

Programmieringsparadigmer 2025

5. kursusgang: Rekursion

Hans Hüttel

7. oktober 2025

Vores plan for i dag

1. Læringsmålene
2. Præsentation af diskussionsopgaverne
3. Opgave: reverse
4. Diskussion: Aftagende lister
5. Opgave: **descending**
6. Pause
7. Diskussion: **isolate**
8. Mikroforelæsning: En pænere definition af **isolate**
9. Opgave: **wrapup**
10. Opgave: **tripler**
11. Hvis tiden tillader det: Flere opgaver i dit eget tempo.
12. Vi evaluerer dagens session – **bliv venligst til slut!**

Læringsmål

- ▶ At forstå strukturen i en rekursiv funktionsdefinition
- ▶ At kunne læse og skrive rekursive funktionsdefinitioner på lister
- ▶ At kunne læse og skrive rekursive definitioner, der bruger multiple rekursioner eller gensidig rekursion
- ▶ At kunne skrive rekursive funktionsdefinitioner på en struktureret måde

Forberedelsesopgave – replicate

Definér funktionen `replicate` – og *brug mønstre* i din løsning. Denne funktion tager et heltal n og et element x og returnerer en liste med n elementer, hvor x er gentaget præcis n gange.

Som eksempel skal `replicate 3 5` give os `[5,5,5]`.

Hvilken type skal `replicate` have?

Forberedelsesopgave – improve

Definér funktionen `improve` – og *brug mønstre* i din løsning. Den tager en liste xs og, hvis xs indeholder mindst to elementer, returnerer den en liste, hvor hvert andet element er fjernet.

Som eksempel skal `improve [1,2,3,4,5,6,7]` give os `[1,3,5,7]` .

Hvilken type skal `improve` have?

Nyt om kurset

- ▶ Vi holder en **prøveeksamen** den 9. december 2025 i 0.1.95, hvor man kan se hvordan eksamen er og teste sig selv.
- ▶ 7. kursusgang finder sted **onsdag den 22. oktober kl. 14.30-16.15 i 0.1.95.**

Om opgaverne

- ▶ Når I arbejder med opgaverne i dette sæt (og i fremtiden), skal I bruge tilgangen beskrevet i afsnit 6.6 i bogen – at lære denne strategi er en måde at nå læringsmålene for denne del af kurset.
- ▶ **Undgå "hovedhaleri"!** (at bruge head, tail osv. til at skille lister ad med) Mønstre er din ven her. Brug dem i stedet.
- ▶ **Undgå "talsanderi"!**, det vil sige lad være med at antage, at hele tal og sandhedsværdier er alt, hvad vi har. Polymorfi er overalt! Lav din typespecifikation **som en kommentar** først og brug typeinferens til at finde den faktiske type. Typer er ofte mere generelle, end man antager.

Opgave 1 – reverse (10 minutter)

Funktionen `reverse` findes i Haskell-præludiet. Den vender en liste om, så f.eks. `reverse [1,2,3]` evalueres til `[3,2,1]`.

Nu er det jeres opgave at definere din egen version af denne funktion, `rev`.

Diskussion – Aftagende lister

En liste er *aftagende*, hvis den er tom eller $[a_1, a_2, \dots, a_n]$ for $n \geq 1$ og $a_1 \geq a_2 \geq \dots \geq a_n$.

Listen `[6,5,5,1]` er aftagende. Listen `["plip","pli","ppp"]` er det ikke.

En berømt stand-up-komiker har defineret en funktion `descending`, der returnerer `True`, hvis en liste er aftagende, og `False` ellers. Her er, hvad komikeren skrev:

```
descending (x:y:xs) = if (x >= y)
                        then True
                        else False
```

Vil dette virke?

Opgave 2 – descending (15 minutter)

Nu er det jeres tur! Skriv en funktion `descending`, der returnerer `True`, hvis en liste er faldende, og `False` ellers.

Pause

Diskussion – isolate

Funktionen `isolate` tager en liste `l` og et element `x` og returnerer et par af to nye lister (`l1`, `l2`). Den første liste `l1` indeholder alle elementer i `l`, der ikke er lig med `x`. Den anden liste `l2` indeholder alle forekomster af `x` i `l`.

- ▶ `isolate [4,5,4,6,7,4] 4` evalueres til `([5,6,7],[4,4,4])`.
- ▶ `isolate ['g','a','k','a'] 'a'` evalueres til `(['g','k'], ['a','a'])`.

En meget selvsikker Java-programmør skrev følgende definition af `isolate` (kan også findes på Moodle-siden som `isolate.hs`) ved at bruge sine Java-færdigheder.

```

isolate ys x = if (head ys == x) == False
                  then ([ (head ys)
                        ++ fst (isolate (tail
                                       ys) x),
                          snd (isolate (
                                       tail ys) x)
                        )
                  else
                  (fst (isolate (tail
                                 ys) x) , [(head ys)
                                           ])
                  ++ (snd (isolate (
                                 tail ys) x)))

```

Java-programmøren udbrød: **Hvorfor skal alt være så kluntet i Haskell?** Kommentarer, tak!

Mikroforelæsning – En pænere udgave af `isolate`

Nu vil jeg definere `isolate` i Haskell ved hjælp af rekursion¹, og *uden* at bruge `fst`, `snd`, `head` eller `tail`.

¹Ikke listeabstraktion – det var sidste uge!

Nogle gode råd

- ▶ Find ud af hvor mange forskellige tilfælde, vi skal håndtere
- ▶ Placer det rekursive kald i en where-klausul
- ▶ Brug mønstermatching til at finde komponenterne i resultatet af det rekursive kald.
- ▶ Husk at mønstre kan have lige så meget struktur, man har brug for.

Opgave 3 – Sammenpakning (20 minutter)

Funktionen `wrapup` er en funktion, der tager en liste og returnerer en liste af lister. Hver liste i denne liste indeholder de på hinanden følgende elementer fra den oprindelige liste, der er identiske. For eksempel skal `wrapup [1,1,1,2,3,3,2]` give os

`[[1,1,1],[2],[3,3],[2]]` .

`wrapup [True,True,False,False,False,True]` skal give os

`[[ True,True],[ False,False,False ],[ True]]`. Definér `wrapup` i

Haskell ved hjælp af rekursion², men *uden* at bruge `fst`, `snd`, `head` eller `tail`. Hvilken type har `wrapup`?

²Ikke listeforståelse – det var sidste uge!

Opgave 4 – Tripler (20 minutter)

En tidligere videnskabs- og uddannelsesminister har besluttet at tage en universitetsuddannelse og forsøger nu at definere en Haskell-funktion `triples`, der tager en liste af tupler (hvert tuppel har præcis 3 elementer) og konverterer denne liste af tupler til en tupel af lister.

`triples [(1,2,3), (4, 5, 6), (7, 8, 9)]` skal give os

`( [1,4,7], [2, 5, 8], [3, 6, 9] )`. Ministeren skrev følgende, men løb ind i problemer. Hvad ser ud til at være galt?

```
triples :: Num a => [(a,a,a)] -> ([a],[a],[a])
```

```
triples [] = ()
```

```
triples [(a,b,c)] = ([a],[b],[c])
```

```
triples (x:xs,y:ys,z:zs) = [x,y,z] : triples [(xs  
    ,ys,zs)]
```

Kan du rette disse fejl? Hvordan kan afsnit 6.6 hjælpe dig her?

Evaluering

- ▶ Hvad fandt I svært?
- ▶ Hvad overraskede jer?
- ▶ Hvad gik godt?